

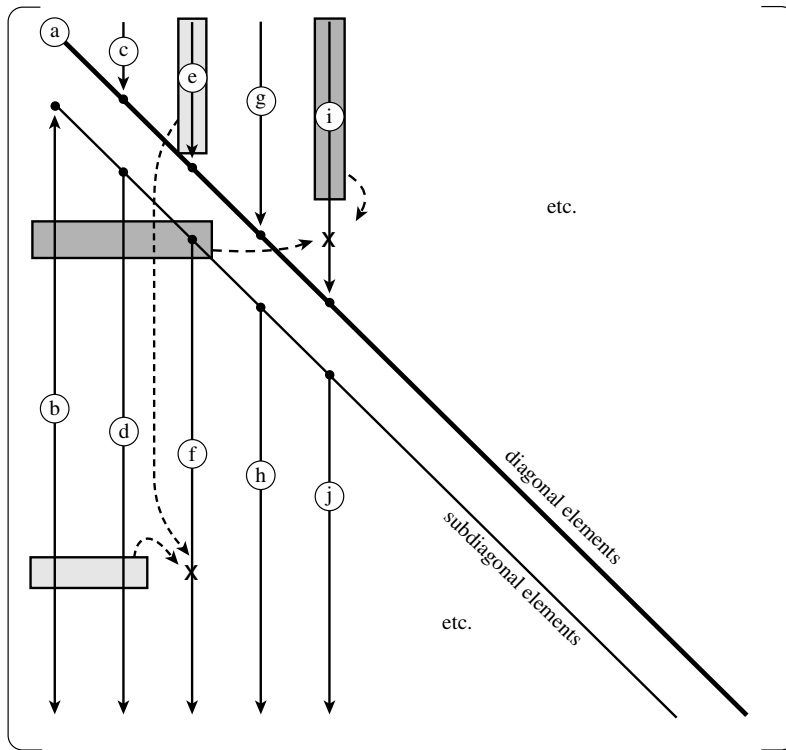


Figure 2.3.1. Crout's algorithm for LU decomposition of a matrix. Elements of the original matrix are modified in the order indicated by lowercase letters: a, b, c, etc. Shaded boxes show the previously modified elements that are used in modifying two typical elements, each indicated by an "x".

element), and then do all the divisions by that element *en masse*. This is *Crout's method with partial pivoting*. Our implementation has one additional wrinkle: It initially finds the largest element in each row, and subsequently (when it is looking for the maximal pivot element) scales the comparison *as if* we had initially scaled all the equations to make their maximum coefficient equal to unity; this is the *implicit pivoting* mentioned in §2.1.

The inner loop of the LU decomposition, equations (2.3.12) and (2.3.13), resembles the inner loop of matrix multiplication. There is a triple loop over the indices i , j , and k . There are six permutations of the order in which these loops can be done. The straightforward implementation of Crout's algorithm corresponds to the jik permutation, where the order of the indices is the order of the loops from outermost to innermost. On modern processors with a hierarchy of cache memory, and when matrices are stored by rows, the fastest execution time is usually the kij or ikj ordering. Pivoting is easier with kij ordering, so that is the implementation we use. This is called "outer product Gaussian elimination" by Golub and Van Loan [1].

LU decomposition is well suited for implementation as an object (a `class` or `struct`). The constructor performs the decomposition, and the object itself stores the result. Then, a method for forward- and backsubstitution can be called once, or many times, to solve for one or more right-hand sides. Methods for additional functionality are also easy to include. The object's declaration looks like this:

ludcmp.h

struct LUdcmp

Object for solving linear equations $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ using LU decomposition, and related functions.

{

Int n;

MatDoub lu;

VecInt indx;

Doub d;

LUdcmp(MatDoub_I &a);

void solve(VecDoub_I &b, VecDoub_O &x);

void solve(MatDoub_I &b, MatDoub_O &x);

void inverse(MatDoub_O &ainv);

Doub det();

void mprove(VecDoub_I &b, VecDoub_IO &x);

MatDoub_I &aref;

};

Stores the decomposition.

Stores the permutation.

Used by det.

Constructor. Argument is the matrix $\mathbf{A}$.

Solve for a single right-hand side.

Solve for multiple right-hand sides.

Calculate matrix inverse $\mathbf{A}^{-1}$.Return determinant of $\mathbf{A}$.

Discussed in §2.5.

Used only by mprove.

Here is the implementation of the constructor, whose argument is the input matrix that is to be LU decomposed. The input matrix is not altered; a copy is made, on which outer product Gaussian elimination is then done in-place.

ludcmp.h

LUdcmp::LUdcmp(MatDoub_I &a) : n(a.nrows()), lu(a), aref(a), indx(n) {

Given a matrix $\mathbf{a}[0..n-1][0..n-1]$, this routine replaces it by the LU decomposition of a rowwise permutation of itself. $\mathbf{a}$ is input. On output, it is arranged as in equation (2.3.14) above; $\text{indx}[0..n-1]$ is an output vector that records the row permutation effected by the partial pivoting; $\mathbf{d}$ is output as ± 1 depending on whether the number of row interchanges was even or odd, respectively. This routine is used in combination with `solve` to solve linear equations or invert a matrix.

const Doub TINY=1.0e-40;

A small number.

Int i,imax,j,k;

Doub big,temp;

VecDoub vv(n);

vv stores the implicit scaling of each row.

d=1.0;

No row interchanges yet.

for (i=0;i<n;i++) {

Loop over rows to get the implicit scaling information.

big=0.0;

for (j=0;j<n;j++)

if ((temp=abs(lu[i][j])) > big) big=temp;

if (big == 0.0) throw("Singular matrix in LUdcmp");

No nonzero largest element.

vv[i]=1.0/big;

Save the scaling.

}

for (k=0;k<n;k++) {

This is the outermost kij loop.

big=0.0;

Initialize for the search for largest pivot element.

for (i=k;i<n;i++) {

temp=vv[i]*abs(lu[i][k]);

if (temp > big) {

Is the figure of merit for the pivot better than the best so far?

big=temp;

imax=i;

}

}

if (k != imax) {

Do we need to interchange rows?

for (j=0;j<n;j++) {

Yes, do so...

temp=lu[imax][j];

lu[imax][j]=lu[k][j];

lu[k][j]=temp;

}

d = -d;

...and change the parity of d.

vv[imax]=vv[k];

Also interchange the scale factor.

}

indx[k]=imax;

if (lu[k][k] == 0.0) lu[k][k]=TINY;

If the pivot element is zero, the matrix is singular (at least to the precision of the algorithm). For some applications on singular matrices, it is desirable to substitute

```

TINY for zero.
for (i=k+1;i<n;i++) {
    temp=lu[i][k] /= lu[k][k]; Divide by the pivot element.
    for (j=k+1;j<n;j++) Innermost loop: reduce remaining submatrix.
        lu[i][j] -= temp*lu[k][j];
    }
}
}
}

```

Once the LUdcmp object is constructed, two functions implementing equations (2.3.6) and (2.3.7) are available for solving linear equations. The first solves a single right-hand side vector **b** for a solution vector **x**. The second simultaneously solves multiple right-hand vectors, arranged as the columns of a matrix **B**. In other words, it calculates the matrix $\mathbf{A}^{-1} \cdot \mathbf{B}$.

```
void LUdcmp::solve(VecDoub_I &b, VecDoub_O &x)
```

ludcmp.h

Solves the set of n linear equations $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ using the stored LU decomposition of **A**. $\mathbf{b}[0..n-1]$ is input as the right-hand side vector **b**, while **x** returns the solution vector **x**; **b** and **x** may reference the same vector, in which case the solution overwrites the input. This routine takes into account the possibility that **b** will begin with many zero elements, so it is efficient for use in matrix inversion.

```

{
    Int i,ii=0,ip,j;
    Doub sum;
    if (b.size() != n || x.size() != n)
        throw("LUdcmp::solve bad sizes");
    for (i=0;i<n;i++) x[i] = b[i];
    for (i=0;i<n;i++) {
        ip=indx[i];
        sum=x[ip];
        x[ip]=x[i];
        if (ii != 0)
            for (j=ii-1;j<i;j++) sum -= lu[i][j]*x[j];
        else if (sum != 0.0)
            ii=i+1;
        x[i]=sum;
    }
    for (i=n-1;i>=0;i--) {
        sum=x[i];
        for (j=i+1;j<n;j++) sum -= lu[i][j]*x[j];
        x[i]=sum/lu[i][i];
    }
}

```

When *ii* is set to a positive value, it will become the index of the first nonvanishing element of **b**. We now do the forward substitution, equation (2.3.6). The only new wrinkle is to unscramble the permutation as we go.

A nonzero element was encountered, so from now on we will have to do the sums in the loop above.

Now we do the backsubstitution, equation (2.3.7).

Store a component of the solution vector **X**.

All done!

```
void LUdcmp::solve(MatDoub_I &b, MatDoub_O &x)
```

Solves m sets of n linear equations $\mathbf{A} \cdot \mathbf{X} = \mathbf{B}$ using the stored LU decomposition of **A**. The matrix $\mathbf{b}[0..n-1][0..m-1]$ inputs the right-hand sides, while $\mathbf{x}[0..n-1][0..m-1]$ returns the solution $\mathbf{A}^{-1} \cdot \mathbf{B}$. **b** and **x** may reference the same matrix, in which case the solution overwrites the input.

```

{
    int i,j,m=b.ncols();
    if (b.nrows() != n || x.nrows() != n || b.ncols() != x.ncols())
        throw("LUdcmp::solve bad sizes");
    VecDoub xx(n);
    for (j=0;j<m;j++) {
        Copy and solve each column in turn.
        for (i=0;i<n;i++) xx[i] = b[i][j];
        solve(xx,xx);
        for (i=0;i<n;i++) x[i][j] = xx[i];
    }
}

```

The LU decomposition in `LUdcmp` requires about $\frac{1}{3}N^3$ executions of the inner loops (each with one multiply and one add). This is thus the operation count for solving one (or a few) right-hand sides, and is a factor of 3 better than the Gauss-Jordan routine `gaussj` that was given in §2.1, and a factor of 1.5 better than a Gauss-Jordan routine (not given) that does not compute the inverse matrix. For inverting a matrix, the total count (including the forward- and backsubstitution as discussed following equation 2.3.7 above) is $(\frac{1}{3} + \frac{1}{6} + \frac{1}{2})N^3 = N^3$, the same as `gaussj`.

To summarize, this is the preferred way to solve the linear set of equations $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$:

```
const Int n = ...
MatDoub a(n,n);
VecDoub b(n),x(n);
...
LUdcmp alu(a);
alu.solve(b,x);
```

The answer will be given back in `x`. Your original matrix `a` and vector `b` are not altered. If you need to recover the storage in the object `alu`, then start a temporary scope with “{” before `alu` is declared, and end that scope with “}” when you want `alu` to be destroyed.

If you subsequently want to solve a set of equations with the same $\mathbf{A}$ but a different right-hand side $\mathbf{b}$, you repeat *only*

```
alu.solve(b,x);
```

2.3.2 Inverse of a Matrix

`LUdcmp` has a member function that gives the inverse of the matrix $\mathbf{A}$. Simply, it creates an identity matrix and then invokes the appropriate `solve` method.

`ludcmp.h`

```
void LUdcmp::inverse(MatDoub_0 &ainv)
```

Using the stored LU decomposition, return in `ainv` the matrix inverse $\mathbf{A}^{-1}$.

```
{
    Int i,j;
    ainv.resize(n,n);
    for (i=0;i<n;i++) {
        for (j=0;j<n;j++) ainv[i][j] = 0.;
        ainv[i][i] = 1.;
    }
    solve(ainv,ainv);
}
```

The matrix `ainv` will now contain the inverse of the original matrix `a`. Alternatively, there is nothing wrong with using a Gauss-Jordan routine like `gaussj` (§2.1) to invert a matrix in place, destroying the original. Both methods have practically the same operations count.

2.3.3 Determinant of a Matrix

The determinant of an LU decomposed matrix is just the product of the diagonal elements,

$$\det = \prod_{j=0}^{N-1} \beta_{jj} \quad (2.3.15)$$

We don't, recall, compute the decomposition of the original matrix, but rather a decomposition of a rowwise permutation of it. Luckily, we have kept track of whether the number of row interchanges was even or odd, so we just preface the product by the corresponding sign. (You now finally know the purpose of `d` in the `LUdcmp` structure.)

`Doub LUdcmp::det()`

`ludcmp.h`

Using the stored *LU* decomposition, return the determinant of the matrix **A**.

```
{
    Doub dd = d;
    for (Int i=0; i<n; i++) dd *= lu[i][i];
    return dd;
}
```

For a matrix of any substantial size, it is quite likely that the determinant will overflow or underflow your computer's floating-point dynamic range. In such a case you can easily add another member function that, e.g., divides by powers of ten, to keep track of the scale separately, or, e.g., accumulates the sum of logarithms of the absolute values of the factors and the sign separately.

2.3.4 Complex Systems of Equations

If your matrix **A** is real, but the right-hand side vector is complex, say **b** + *i***d**, then (i) *LU* decompose **A** in the usual way, (ii) backsubstitute **b** to get the real part of the solution vector, and (iii) backsubstitute **d** to get the imaginary part of the solution vector.

If the matrix itself is complex, so that you want to solve the system

$$(\mathbf{A} + i\mathbf{C}) \cdot (\mathbf{x} + i\mathbf{y}) = (\mathbf{b} + i\mathbf{d}) \quad (2.3.16)$$

then there are two possible ways to proceed. The best way is to rewrite `LUdcmp` with complex routines. Complex modulus substitutes for absolute value in the construction of the scaling vector `vv` and in the search for the largest pivot elements. Everything else goes through in the obvious way, with complex arithmetic used as needed.

A quick-and-dirty way to solve complex systems is to take the real and imaginary parts of (2.3.16), giving

$$\begin{aligned} \mathbf{A} \cdot \mathbf{x} - \mathbf{C} \cdot \mathbf{y} &= \mathbf{b} \\ \mathbf{C} \cdot \mathbf{x} + \mathbf{A} \cdot \mathbf{y} &= \mathbf{d} \end{aligned} \quad (2.3.17)$$

which can be written as a $2N \times 2N$ set of *real* equations,

$$\begin{pmatrix} \mathbf{A} & -\mathbf{C} \\ \mathbf{C} & \mathbf{A} \end{pmatrix} \cdot \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \end{pmatrix} = \begin{pmatrix} \mathbf{b} \\ \mathbf{d} \end{pmatrix} \quad (2.3.18)$$

and then solved with `LUdcmp`'s routines in their present forms. This scheme is a factor of 2 inefficient in storage, since **A** and **C** are stored twice. It is also a factor of 2 inefficient in time, since the complex multiplies in a complexified version of the routines would each use 4 real multiplies, while the solution of a $2N \times 2N$ problem involves 8 times the work of an $N \times N$ one. If you can tolerate these factor-of-two inefficiencies, then equation (2.3.18) is an easy way to proceed.

CITED REFERENCES AND FURTHER READING:

Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), Chapter 4.[1]

- Anderson, E., et al. 1999, LAPACK User's Guide, 3rd ed. (Philadelphia: S.I.A.M.). Online with software at 2007+, <http://www.netlib.org/lapack>.
- Forsythe, G.E., Malcolm, M.A., and Moler, C.B. 1977, *Computer Methods for Mathematical Computations* (Englewood Cliffs, NJ: Prentice-Hall), §3.3, and p. 50.
- Forsythe, G.E., and Moler, C.B. 1967, *Computer Solution of Linear Algebraic Systems* (Englewood Cliffs, NJ: Prentice-Hall), Chapters 9, 16, and 18.
- Westlake, J.R. 1968, *A Handbook of Numerical Matrix Inversion and Solution of Linear Equations* (New York: Wiley).
- Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §4.1.
- Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed.; reprinted 2001 (New York: Dover), §9.11.
- Horn, R.A., and Johnson, C.R. 1985, *Matrix Analysis* (Cambridge: Cambridge University Press).

2.4 Tridiagonal and Band-Diagonal Systems of Equations

The special case of a system of linear equations that is *tridiagonal*, that is, has nonzero elements only on the diagonal plus or minus one column, is one that occurs frequently. Also common are systems that are *band-diagonal*, with nonzero elements only along a few diagonal lines adjacent to the main diagonal (above and below).

For tridiagonal sets, the procedures of *LU* decomposition, forward- and back-substitution each take only $O(N)$ operations, and the whole solution can be encoded very concisely. The resulting routine `tridag` is one that we will use in later chapters.

Naturally, one does not reserve storage for the full $N \times N$ matrix, but only for the nonzero components, stored as three vectors. The set of equations to be solved is

$$\begin{bmatrix} b_0 & c_0 & 0 & \cdots \\ a_1 & b_1 & c_1 & \cdots \\ & & \cdots & \\ & & \cdots & a_{N-2} & b_{N-2} & c_{N-2} \\ & & \cdots & 0 & a_{N-1} & b_{N-1} \end{bmatrix} \cdot \begin{bmatrix} u_0 \\ u_1 \\ \cdots \\ u_{N-2} \\ u_{N-1} \end{bmatrix} = \begin{bmatrix} r_0 \\ r_1 \\ \cdots \\ r_{N-2} \\ r_{N-1} \end{bmatrix} \quad (2.4.1)$$

Notice that a_0 and c_{N-1} are undefined and are not referenced by the routine that follows.

```
tridag.h void tridag(VecDoub_I &a, VecDoub_I &b, VecDoub_I &c, VecDoub_I &r, VecDoub_O &u)
Solves for a vector u[0..n-1] the tridiagonal linear set given by equation (2.4.1). a[0..n-1],
b[0..n-1], c[0..n-1], and r[0..n-1] are input vectors and are not modified.
{
    Int j,n=a.size();
    Doub bet;
    VecDoub gam(n);           One vector of workspace, gam, is needed.
    if (b[0] == 0.0) throw("Error 1 in tridag");
    If this happens, then you should rewrite your equations as a set of order N - 1, with u1
    trivially eliminated.
    u[0]=r[0]/(bet=b[0]);
    for (j=1;j<n;j++) {        Decomposition and forward substitution.
        gam[j]=c[j-1]/bet;
        bet=b[j]-a[j]*gam[j];
    }
```

```

    if (bet == 0.0) throw("Error 2 in tridag");      Algorithm fails; see below.
    u[j]=(r[j]-a[j]*u[j-1])/bet;
  }
  for (j=(n-2);j>=0;j--)
    u[j] -= gam[j+1]*u[j+1];      Backsubstitution.
}

```

There is no pivoting in `tridag`. It is for this reason that `tridag` can fail even when the underlying matrix is nonsingular: A zero pivot can be encountered even for a nonsingular matrix. In practice, this is not something to lose sleep about. The kinds of problems that lead to tridiagonal linear sets usually have additional properties which guarantee that the algorithm in `tridag` will succeed. For example, if

$$|b_j| > |a_j| + |c_j| \quad j = 0, \dots, N-1 \quad (2.4.2)$$

(called *diagonal dominance*), then it can be shown that the algorithm cannot encounter a zero pivot.

It is possible to construct special examples in which the lack of pivoting in the algorithm causes numerical instability. In practice, however, such instability is almost never encountered — unlike the general matrix problem where pivoting is essential.

The tridiagonal algorithm is the rare case of an algorithm that, in practice, is more robust than theory says it should be. Of course, should you ever encounter a problem for which `tridag` fails, you can instead use the more general method for band-diagonal systems, described below (the `Bandec` object).

Some other matrix forms consisting of tridiagonal with a small number of additional elements (e.g., upper right and lower left corners) also allow rapid solution; see §2.7.

2.4.1 Parallel Solution of Tridiagonal Systems

It is possible to solve tridiagonal systems doing many of the operations in parallel. We illustrate by the special case with $N = 7$:

$$\begin{bmatrix} b_0 & c_0 & & & & & \\ a_1 & b_1 & c_1 & & & & \\ & a_2 & b_2 & c_2 & & & \\ & & a_3 & b_3 & c_3 & & \\ & & & a_4 & b_4 & c_4 & \\ & & & & a_5 & b_5 & c_5 \\ & & & & & a_6 & b_6 \end{bmatrix} \cdot \begin{bmatrix} u_0 \\ u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \end{bmatrix} = \begin{bmatrix} r_0 \\ r_1 \\ r_2 \\ r_3 \\ r_4 \\ r_5 \\ r_6 \end{bmatrix} \quad (2.4.3)$$

The basic idea is to partition the problem into even and odd elements, recurse to solve the latter, and then solve the former in parallel. Specifically, we first rewrite equation (2.4.3) by permuting its rows and columns,

$$\begin{bmatrix} b_0 & & & c_0 & & & \\ & b_2 & & a_2 & c_2 & & \\ & & b_4 & & a_4 & c_4 & \\ & & & b_6 & & a_6 & \\ a_1 & c_1 & & & b_1 & & \\ & a_3 & c_3 & & & b_3 & \\ & & a_5 & c_5 & & & b_5 \end{bmatrix} \cdot \begin{bmatrix} u_0 \\ u_2 \\ u_4 \\ u_6 \\ u_1 \\ u_3 \\ u_5 \end{bmatrix} = \begin{bmatrix} r_0 \\ r_2 \\ r_4 \\ r_6 \\ r_1 \\ r_3 \\ r_5 \end{bmatrix} \quad (2.4.4)$$

Now observe that, by row operations that subtract multiples of the first four rows from each of the last three rows, we can eliminate all nonzero elements in the lower-left quadrant. The price we pay is bringing some new elements into the lower-right quadrant, whose

nonzero elements we now call x 's, y 's, and z 's. We call the modified right-hand sides q . The transformed problem is now

$$\begin{bmatrix} b_0 & & & c_0 & & \\ & b_2 & & a_2 & c_2 & \\ & & b_4 & & a_4 & c_4 \\ & & & b_6 & & a_6 \\ & & & & y_0 & z_0 \\ & & & & x_1 & y_1 & z_1 \\ & & & & & x_2 & y_2 \end{bmatrix} \cdot \begin{bmatrix} u_0 \\ u_2 \\ u_4 \\ u_6 \\ u_1 \\ u_3 \\ u_5 \end{bmatrix} = \begin{bmatrix} r_0 \\ r_2 \\ r_4 \\ r_6 \\ q_0 \\ q_1 \\ q_2 \end{bmatrix} \quad (2.4.5)$$

Notice that the last three rows form a new, smaller, tridiagonal problem, which we can solve simply by recursing. Once its solution is known, the first four rows can be solved by a simple, parallelizable, substitution. For discussion of this and related methods for parallelizing tridiagonal systems, and references to the literature, see [2].

2.4.2 Band-Diagonal Systems

Where tridiagonal systems have nonzero elements only on the diagonal plus or minus one, band-diagonal systems are slightly more general and have (say) $m_1 \geq 0$ nonzero elements immediately to the left of (below) the diagonal and $m_2 \geq 0$ nonzero elements immediately to its right (above it). Of course, this is only a useful classification if m_1 and m_2 are both $\ll N$. In that case, the solution of the linear system by LU decomposition can be accomplished much faster, and in much less storage, than for the general $N \times N$ case.

The precise definition of a band-diagonal matrix with elements a_{ij} is that

$$a_{ij} = 0 \quad \text{when} \quad j > i + m_2 \quad \text{or} \quad i > j + m_1 \quad (2.4.6)$$

Band-diagonal matrices are stored and manipulated in a so-called compact form, which results if the matrix is tilted 45° clockwise, so that its nonzero elements lie in a long, narrow matrix with $m_1 + 1 + m_2$ columns and N rows. This is best illustrated by an example: The band-diagonal matrix

$$\begin{pmatrix} 3 & 1 & 0 & 0 & 0 & 0 & 0 \\ 4 & 1 & 5 & 0 & 0 & 0 & 0 \\ 9 & 2 & 6 & 5 & 0 & 0 & 0 \\ 0 & 3 & 5 & 8 & 9 & 0 & 0 \\ 0 & 0 & 7 & 9 & 3 & 2 & 0 \\ 0 & 0 & 0 & 3 & 8 & 4 & 6 \\ 0 & 0 & 0 & 0 & 2 & 4 & 4 \end{pmatrix} \quad (2.4.7)$$

which has $N = 7$, $m_1 = 2$, and $m_2 = 1$, is stored compactly as the 7×4 matrix,

$$\begin{pmatrix} x & x & 3 & 1 \\ x & 4 & 1 & 5 \\ 9 & 2 & 6 & 5 \\ 3 & 5 & 8 & 9 \\ 7 & 9 & 3 & 2 \\ 3 & 8 & 4 & 6 \\ 2 & 4 & 4 & x \end{pmatrix} \quad (2.4.8)$$

Here x denotes elements that are wasted space in the compact format; these will not be referenced by any manipulations and can have arbitrary values. Notice that the diagonal of the original matrix appears in column m_1 , with subdiagonal elements to its left and superdiagonal elements to its right.

The simplest manipulation of a band-diagonal matrix, stored compactly, is to multiply it by a vector to its right. Although this is algorithmically trivial, you might want to study the following routine as an example of how to pull nonzero elements a_{ij} out of the compact storage format in an orderly fashion.


```
void banmul(MatDoub_I &a, const Int m1, const Int m2, VecDoub_I &x,
           VecDoub_O &b)
```

banded.h

Matrix multiply $\mathbf{b} = \mathbf{A} \cdot \mathbf{x}$, where $\mathbf{A}$ is band-diagonal with m_1 rows below the diagonal and m_2 rows above. The input vector is $\mathbf{x}[0..n-1]$ and the output vector is $\mathbf{b}[0..n-1]$. The array $\mathbf{a}[0..n-1][0..m_1+m_2]$ stores $\mathbf{A}$ as follows: The diagonal elements are in $\mathbf{a}[0..n-1][m_1]$. Subdiagonal elements are in $\mathbf{a}[j..n-1][0..m_1-1]$ with $j > 0$ appropriate to the number of elements on each subdiagonal. Superdiagonal elements are in $\mathbf{a}[0..j][m_1+1..m_1+m_2]$ with $j < n-1$ appropriate to the number of elements on each superdiagonal.

```
{
    Int i,j,k,tmploop,n=a.nrows();
    for (i=0;i<n;i++) {
        k=i-m1;
        tmploop=MIN(m1+m2+1,Int(n-k));
        b[i]=0.0;
        for (j=MAX(0,-k);j<tmploop;j++) b[i] += a[i][j]*x[j+k];
    }
}
```

It is not possible to store the LU decomposition of a band-diagonal matrix $\mathbf{A}$ quite as compactly as the compact form of $\mathbf{A}$ itself. The decomposition (essentially by Crout's method; see §2.3) produces additional nonzero "fill-ins." One straightforward storage scheme is to store the upper triangular factor (U) in a space with the same shape as $\mathbf{A}$, and to store the lower triangular factor (L) in a separate compact matrix of size $N \times m_1$. The diagonal elements of U (whose product, times $d = \pm 1$, gives the determinant) are in the first column of U .

Here is an object, analogous to `LUdcmp` in §2.3, for solving band-diagonal linear equations:

```
struct Bandedc {
```

banded.h

Object for solving linear equations $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ for a band-diagonal matrix $\mathbf{A}$, using LU decomposition.

```
    Int n,m1,m2;
    MatDoub au,al;           Upper and lower triangular matrices, stored compactly.
    VecInt indx;
    Doub d;
    Bandedc(MatDoub_I &a, const int mm1, const int mm2); Constructor.
    void solve(VecDoub_I &b, VecDoub_O &x); Solve a right-hand side vector.
    Doub det(); Return determinant of A.
};
```

The constructor takes as arguments the compactly stored matrix $\mathbf{A}$, and the integers m_1 and m_2 . (One could of course define a "band-diagonal matrix object" to encapsulate these quantities, but in this brief treatment we want to keep things simple.)

```
Bandedc::Bandedc(MatDoub_I &a, const Int mm1, const Int mm2)
: n(a.nrows()), au(a), m1(mm1), m2(mm2), al(n,m1), indx(n)
```

banded.h

Constructor. Given an $n \times n$ band-diagonal matrix $\mathbf{A}$ with m_1 subdiagonal rows and m_2 superdiagonal rows, compactly stored in the array $\mathbf{a}[0..n-1][0..m_1+m_2]$ as described in the comment for routine `banmul`, an LU decomposition of a rowwise permutation of $\mathbf{A}$ is constructed. The upper and lower triangular matrices are stored in `au` and `al`, respectively. The stored vector `indx[0..n-1]` records the row permutation effected by the partial pivoting; d is ± 1 depending on whether the number of row interchanges was even or odd, respectively.

```
{
    const Doub TINY=1.0e-40;
    Int i,j,k,l,mm;
    Doub dum;
    mm=m1+m2+1;
    l=m1;
    for (i=0;i<m1;i++) { Rearrange the storage a bit.
        for (j=m1-i;j<mm;j++) au[i][j-1]=au[i][j];
        l--;
        for (j=mm-l-1;j<mm;j++) au[i][j]=0.0;
    }
}
```

```

d=1.0;
l=m1;
for (k=0;k<n;k++) {                                For each row...
    dum=au[k][0];
    i=k;
    if (l<n) l++;
    for (j=k+1;j<l;j++) {                            Find the pivot element.
        if (abs(au[j][0]) > abs(dum)) {
            dum=au[j][0];
            i=j;
        }
    }
    indx[k]=i+1;
    if (dum == 0.0) au[k][0]=TINY;
    Matrix is algorithmically singular, but proceed anyway with TINY pivot (desirable in
    some applications).
    if (i != k) {                                    Interchange rows.
        d = -d;
        for (j=0;j<mm;j++) SWAP(au[k][j],au[i][j]);
    }
    for (i=k+1;i<l;i++) {                            Do the elimination.
        dum=au[i][0]/au[k][0];
        al[k][i-k-1]=dum;
        for (j=1;j<mm;j++) au[i][j-1]=au[i][j]-dum*au[k][j];
        au[i][mm-1]=0.0;
    }
}
}

```

Some pivoting is possible within the storage limitations of `bandec`, and the above routine does take advantage of the opportunity. In general, when `TINY` is returned as a diagonal element of U , then the original matrix (perhaps as modified by roundoff error) is in fact singular. In this regard, `bandec` is somewhat more robust than `tridag` above, which can fail algorithmically even for nonsingular matrices; `bandec` is thus also useful (with $m_1 = m_2 = 1$) for some ill-behaved tridiagonal systems.

Once the matrix A has been decomposed, any number of right-hand sides can be solved in turn by repeated calls to the `solve` method, the forward- and backsubstitution routine analogous to its same-named cousin in §2.3.

`banded.h`

```

void Bandec::solve(VecDoub_I &b, VecDoub_O &x)
Given a right-hand side vector b[0..n-1], solves the band-diagonal linear equations  $A \cdot x = b$ .
The solution vector  $x$  is returned as x[0..n-1].
{
    Int i,j,k,l,mm;
    Doub dum;
    mm=m1+m2+1;
    l=m1;
    for (k=0;k<n;k++) x[k] = b[k];
    for (k=0;k<n;k++) {                                Forward substitution, unscrambling the permuted rows
        j=indx[k]-1;                                    as we go.
        if (j!=k) SWAP(x[k],x[j]);
        if (l<n) l++;
        for (j=k+1;j<l;j++) x[j] -= al[k][j-k-1]*x[k];
    }
    l=1;
    for (i=n-1;i>=0;i--) {                            Backsubstitution.
        dum=x[i];
        for (k=1;k<l;k++) dum -= au[i][k]*x[k+i];
        x[i]=dum/au[i][0];
        if (l<mm) l++;
    }
}

```

And, finally, a method for getting the determinant:

```
Doub Bandec::det() {
    Using the stored decomposition, return the determinant of the matrix A.
    Doub dd = d;
    for (int i=0; i<n; i++) dd *= au[i][0];
    return dd;
}
```

banded.h

The routines in Bandec are based on the Handbook routines *bandet1* and *bansol1* in [1].

CITED REFERENCES AND FURTHER READING:

- Keller, H.B. 1968, *Numerical Methods for Two-Point Boundary-Value Problems*; reprinted 1991 (New York: Dover), p. 74.
- Dahlquist, G., and Bjorck, A. 1974, *Numerical Methods* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2003 (New York: Dover), Example 5.4.3, p. 166.
- Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed.; reprinted 2001 (New York: Dover), §9.11.
- Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer), Chapter I/6.[1]
- Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §4.3.
- Hockney, R.W., and Jesshope, C.R. 1988, *Parallel Computers 2: Architecture, Programming, and Algorithms* (Bristol and Philadelphia: Adam Hilger), §5.4.[2]

2.5 Iterative Improvement of a Solution to Linear Equations

Obviously it is not easy to obtain greater precision for the solution of a linear set than the precision of your computer's floating-point word. Unfortunately, for large sets of linear equations, it is not always easy to obtain precision equal to, or even comparable to, the computer's limit. In direct methods of solution, roundoff errors accumulate, and they are magnified to the extent that your matrix is close to singular. You can easily lose two or three significant figures for matrices that (you thought) were *far* from singular.

If this happens to you, there is a neat trick to restore the full machine precision, called *iterative improvement* of the solution. The theory is straightforward (see Figure 2.5.1): Suppose that a vector $\mathbf{x}$ is the exact solution of the linear set

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b} \quad (2.5.1)$$

You don't, however, know $\mathbf{x}$. You only know some slightly wrong solution $\mathbf{x} + \delta\mathbf{x}$, where $\delta\mathbf{x}$ is the unknown error. When multiplied by the matrix $\mathbf{A}$, your slightly wrong solution gives a product slightly discrepant from the desired right-hand side $\mathbf{b}$, namely

$$\mathbf{A} \cdot (\mathbf{x} + \delta\mathbf{x}) = \mathbf{b} + \delta\mathbf{b} \quad (2.5.2)$$

Subtracting (2.5.1) from (2.5.2) gives

$$\mathbf{A} \cdot \delta\mathbf{x} = \delta\mathbf{b} \quad (2.5.3)$$

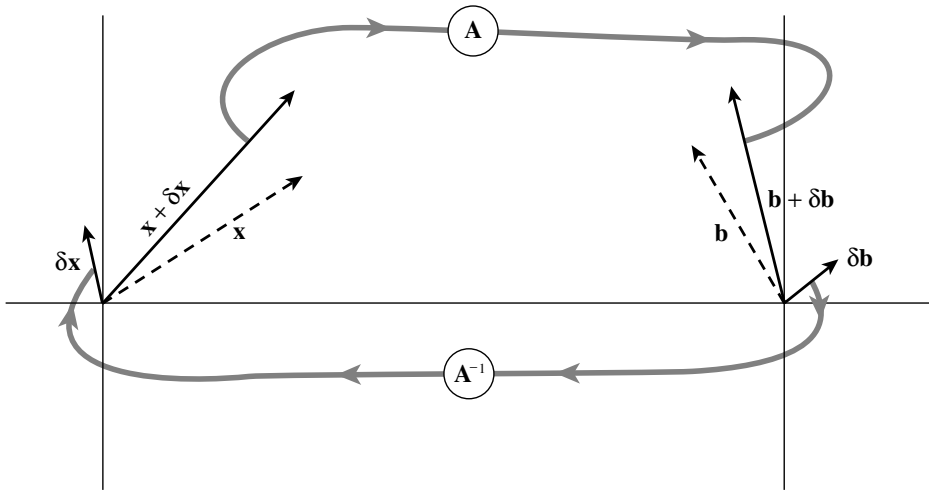


Figure 2.5.1. Iterative improvement of the solution to $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$. The first guess $\mathbf{x} + \delta\mathbf{x}$ is multiplied by $\mathbf{A}$ to produce $\mathbf{b} + \delta\mathbf{b}$. The known vector $\mathbf{b}$ is subtracted, giving $\delta\mathbf{b}$. The linear set with this right-hand side is inverted, giving $\delta\mathbf{x}$. This is subtracted from the first guess giving an improved solution $\mathbf{x}$.

But (2.5.2) can also be solved, trivially, for $\delta\mathbf{b}$. Substituting this into (2.5.3) gives

$$\mathbf{A} \cdot \delta\mathbf{x} = \mathbf{A} \cdot (\mathbf{x} + \delta\mathbf{x}) - \mathbf{b} \quad (2.5.4)$$

In this equation, the whole right-hand side is known, since $\mathbf{x} + \delta\mathbf{x}$ is the wrong solution that you want to improve. It is good to calculate the right-hand side in higher precision than the original solution, if you can, since there will be a lot of cancellation in the subtraction of $\mathbf{b}$. Then, we need only solve (2.5.4) for the error $\delta\mathbf{x}$, and then subtract this from the wrong solution to get an improved solution.

An important extra benefit occurs if we obtained the original solution by *LU* decomposition. In this case we already have the *LU* decomposed form of $\mathbf{A}$, and all we need do to solve (2.5.4) is compute the right-hand side, forward- and backsubstitute.

Because so much of the necessary machinery is already in *LUdcmp*, we implement iterative improvement as a member function of that class. Since iterative improvement requires the matrix $\mathbf{A}$ (as well as its *LU* decomposition), we have, with foresight, caused *LUdcmp* to save a reference to the matrix *a* from which it was constructed. If you plan to use iterative improvement, you must not modify *a* or let it go out of scope. (No other method in *LUdcmp* makes use of this reference to *a*.)

ludcmp.h

```
void LUdcmp::mprove(VecDoub_I &b, VecDoub_IO &x)
```

Improves a solution vector $\mathbf{x}[0..n-1]$ of the linear set of equations $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$. The vectors $\mathbf{b}[0..n-1]$ and $\mathbf{x}[0..n-1]$ are input. On output, $\mathbf{x}[0..n-1]$ is modified, to an improved set of values.

```
{
    Int i,j;
    VecDoub r(n);
    for (i=0;i<n;i++) {
        Ldoub sdp = -b[i];
        for (j=0;j<n;j++)
            sdp += (Ldoub)aref[i][j] * (Ldoub)x[j];
```

Calculate the right-hand side, accumulating the residual in higher precision.

```

    r[i]=sdp;
}
solve(r,r);
for (i=0;i<n;i++) x[i] -= r[i];
}

```

Solve for the error term,
and subtract it from the old solution.

Iterative improvement is *highly* recommended: It is a process of order only N^2 operations (multiply vector by matrix, forward- and backsubstitute — see discussion following equation 2.3.7); it never hurts; and it can really give you your money's worth if it saves an otherwise ruined solution on which you have already spent of order N^3 operations.

You can call `improve` several times in succession if you want. Unless you are starting quite far from the true solution, one call is generally enough; but a second call to verify convergence can be reassuring.

If you cannot compute the right-hand side in equation (2.5.4) in higher precision, iterative refinement will still often improve the quality of a solution, although not in all cases as much as if higher precision is available. Many textbooks assert the contrary, but you will find the proof in [1].

2.5.1 More on Iterative Improvement

It is illuminating (and will be useful later in the book) to give a somewhat more solid analytical foundation for equation (2.5.4), and also to give some additional results. Implicit in the previous discussion was the notion that the solution vector $\mathbf{x} + \delta\mathbf{x}$ has an error term; but we neglected the fact that the LU decomposition of $\mathbf{A}$ is itself not exact.

A different analytical approach starts with some matrix $\mathbf{B}_0$ that is assumed to be an *approximate* inverse of the matrix $\mathbf{A}$, so that $\mathbf{B}_0 \cdot \mathbf{A}$ is approximately the identity matrix $\mathbf{1}$. Define the *residual matrix* $\mathbf{R}$ of $\mathbf{B}_0$ as

$$\mathbf{R} \equiv \mathbf{1} - \mathbf{B}_0 \cdot \mathbf{A} \quad (2.5.5)$$

which is supposed to be “small” (we will be more precise below). Note that therefore

$$\mathbf{B}_0 \cdot \mathbf{A} = \mathbf{1} - \mathbf{R} \quad (2.5.6)$$

Next consider the following formal manipulation:

$$\begin{aligned} \mathbf{A}^{-1} &= \mathbf{A}^{-1} \cdot (\mathbf{B}_0^{-1} \cdot \mathbf{B}_0) = (\mathbf{A}^{-1} \cdot \mathbf{B}_0^{-1}) \cdot \mathbf{B}_0 = (\mathbf{B}_0 \cdot \mathbf{A})^{-1} \cdot \mathbf{B}_0 \\ &= (\mathbf{1} - \mathbf{R})^{-1} \cdot \mathbf{B}_0 = (\mathbf{1} + \mathbf{R} + \mathbf{R}^2 + \mathbf{R}^3 + \cdots) \cdot \mathbf{B}_0 \end{aligned} \quad (2.5.7)$$

We can define the n th partial sum of the last expression by

$$\mathbf{B}_n \equiv (\mathbf{1} + \mathbf{R} + \cdots + \mathbf{R}^n) \cdot \mathbf{B}_0 \quad (2.5.8)$$

so that $\mathbf{B}_\infty \rightarrow \mathbf{A}^{-1}$, if the limit exists.

It now is straightforward to verify that equation (2.5.8) satisfies some interesting recurrence relations. As regards solving $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$, where $\mathbf{x}$ and $\mathbf{b}$ are vectors, define

$$\mathbf{x}_n \equiv \mathbf{B}_n \cdot \mathbf{b} \quad (2.5.9)$$

Then it is easy to show that

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{B}_0 \cdot (\mathbf{b} - \mathbf{A} \cdot \mathbf{x}_n) \quad (2.5.10)$$

This is immediately recognizable as equation (2.5.4), with $-\delta\mathbf{x} = \mathbf{x}_{n+1} - \mathbf{x}_n$, and with $\mathbf{B}_0$ taking the role of $\mathbf{A}^{-1}$. We see, therefore, that equation (2.5.4) does not require that the LU decomposition of $\mathbf{A}$ be exact, but only that the implied residual $\mathbf{R}$ be small. In rough terms,

if the residual is smaller than the square root of your computer's roundoff error, then after one application of equation (2.5.10) (that is, going from $\mathbf{x}_0 \equiv \mathbf{B}_0 \cdot \mathbf{b}$ to $\mathbf{x}_1$) the first neglected term, of order $\mathbf{R}^2$, will be smaller than the roundoff error. Equation (2.5.10), like equation (2.5.4), moreover, can be applied more than once, since it uses only $\mathbf{B}_0$, and not any of the higher $\mathbf{B}$'s.

A much more surprising recurrence that follows from equation (2.5.8) is one that more than *doubles* the order n at each stage:

$$\mathbf{B}_{2n+1} = 2\mathbf{B}_n - \mathbf{B}_n \cdot \mathbf{A} \cdot \mathbf{B}_n \quad n = 0, 1, 3, 7, \dots \quad (2.5.11)$$

Repeated application of equation (2.5.11), from a suitable starting matrix $\mathbf{B}_0$, converges *quadratically* to the unknown inverse matrix $\mathbf{A}^{-1}$ (see §9.4 for the definition of “quadratically”). Equation (2.5.11) goes by various names, including *Schultz's Method* and *Hotelling's Method*; see Pan and Reif [2] for references. In fact, equation (2.5.11) is simply the iterative Newton-Raphson method of root finding (§9.4) applied to matrix inversion.

Before you get too excited about equation (2.5.11), however, you should notice that it involves two full matrix multiplications at each iteration. Each matrix multiplication involves N^3 adds and multiplies. But we already saw in §2.1 – §2.3 that direct inversion of $\mathbf{A}$ requires only N^3 adds and N^3 multiplies *in toto*. Equation (2.5.11) is therefore practical only when special circumstances allow it to be evaluated much more rapidly than is the case for general matrices. We will meet such circumstances later, in §13.10.

In the spirit of delayed gratification, let us nevertheless pursue the two related issues: When does the series in equation (2.5.7) converge; and what is a suitable initial guess $\mathbf{B}_0$ (if, for example, an initial *LU* decomposition is not feasible)?

We can define the norm of a matrix as the largest amplification of length that it is able to induce on a vector,

$$\|\mathbf{R}\| \equiv \max_{\mathbf{v} \neq 0} \frac{|\mathbf{R} \cdot \mathbf{v}|}{|\mathbf{v}|} \quad (2.5.12)$$

If we let equation (2.5.7) act on some arbitrary right-hand side $\mathbf{b}$, as one wants a matrix inverse to do, it is obvious that a sufficient condition for convergence is

$$\|\mathbf{R}\| < 1 \quad (2.5.13)$$

Pan and Reif [2] point out that a suitable initial guess for $\mathbf{B}_0$ is any sufficiently small constant ϵ times the matrix transpose of $\mathbf{A}$, that is,

$$\mathbf{B}_0 = \epsilon \mathbf{A}^T \quad \text{or} \quad \mathbf{R} = \mathbf{1} - \epsilon \mathbf{A}^T \cdot \mathbf{A} \quad (2.5.14)$$

To see why this is so involves concepts from Chapter 11; we give here only the briefest sketch: $\mathbf{A}^T \cdot \mathbf{A}$ is a symmetric, positive-definite matrix, so it has real, positive eigenvalues. In its diagonal representation, $\mathbf{R}$ takes the form

$$\mathbf{R} = \text{diag}(1 - \epsilon\lambda_0, 1 - \epsilon\lambda_1, \dots, 1 - \epsilon\lambda_{N-1}) \quad (2.5.15)$$

where all the λ_i 's are positive. Evidently any ϵ satisfying $0 < \epsilon < 2/(\max_i \lambda_i)$ will give $\|\mathbf{R}\| < 1$. It is not difficult to show that the optimal choice for ϵ , giving the most rapid convergence for equation (2.5.11), is

$$\epsilon = 2/(\max_i \lambda_i + \min_i \lambda_i) \quad (2.5.16)$$

Rarely does one know the eigenvalues of $\mathbf{A}^T \cdot \mathbf{A}$ in equation (2.5.16). Pan and Reif derive several interesting bounds, which are computable directly from $\mathbf{A}$. The following choices guarantee the convergence of $\mathbf{B}_n$ as $n \rightarrow \infty$:

$$\epsilon \leq 1 / \sum_{j,k} a_{jk}^2 \quad \text{or} \quad \epsilon \leq 1 / \left(\max_i \sum_j |a_{ij}| \times \max_j \sum_i |a_{ij}| \right) \quad (2.5.17)$$

The latter expression is truly a remarkable formula, which Pan and Reif derive by noting that the vector norm in equation (2.5.12) need not be the usual L_2 norm, but can instead be either the L_∞ (max) norm, or the L_1 (absolute value) norm. See their work for details.

Another approach, with which we have had some success, is to estimate the largest eigenvalue statistically, by calculating $s_i \equiv |\mathbf{A} \cdot \mathbf{v}_i|^2$ for several unit vector $\mathbf{v}_i$'s with randomly chosen directions in N -space. The largest eigenvalue λ can then be bounded by the maximum of $2 \max s_i$ and $2N \text{Var}(s_i)/\mu(s_i)$, where Var and μ denote the sample variance and mean, respectively.

CITED REFERENCES AND FURTHER READING:

- Johnson, L.W., and Riess, R.D. 1982, *Numerical Analysis*, 2nd ed. (Reading, MA: Addison-Wesley), §2.3.4, p. 55.
- Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §3.5.3.
- Dahlquist, G., and Björck, A. 1974, *Numerical Methods* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2003 (New York: Dover), §5.5.6, p. 183.
- Forsythe, G.E., and Moler, C.B. 1967, *Computer Solution of Linear Algebraic Systems* (Englewood Cliffs, NJ: Prentice-Hall), Chapter 13.
- Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed.; reprinted 2001 (New York: Dover), §9.5, p. 437.
- Higham, N.J. 1997, "Iterative Refinement for Linear Systems and LAPACK," *IMA Journal of Numerical Analysis*, vol. 17, pp. 495–509.[1]
- Pan, V., and Reif, J. 1985, "Efficient Parallel Solution of Linear Systems," in *Proceedings of the Seventeenth Annual ACM Symposium on Theory of Computing* (New York: Association for Computing Machinery).[2]

2.6 Singular Value Decomposition

There exists a very powerful set of techniques for dealing with sets of equations or matrices that are either singular or else numerically very close to singular. In many cases where Gaussian elimination and LU decomposition fail to give satisfactory results, this set of techniques, known as *singular value decomposition*, or *SVD*, will diagnose for you precisely what the problem is. In some cases, SVD will not only diagnose the problem, it will also solve it, in the sense of giving you a useful numerical answer, although, as we shall see, not necessarily "the" answer that you thought you should get.

SVD is also the method of choice for solving most *linear least-squares* problems. We will outline the relevant theory in this section, but defer detailed discussion of the use of SVD in this application to Chapter 15, whose subject is the parametric modeling of data.

SVD methods are based on the following theorem of linear algebra, whose proof is beyond our scope: Any $M \times N$ matrix $\mathbf{A}$ can be written as the product of an $M \times N$ column-orthogonal matrix $\mathbf{U}$, an $N \times N$ diagonal matrix $\mathbf{W}$ with positive or zero elements (the *singular values*), and the transpose of an $N \times N$ orthogonal matrix $\mathbf{V}$. The various shapes of these matrices are clearer when shown as tableaux. If $M > N$ (which corresponds to the *overdetermined* situation of more equations than

by Forsythe et al. [2], which is in turn based on the original routine of Golub and Reinsch, found, in various forms, in [4-6] and elsewhere. These references include extensive discussion of the algorithm used. As much as we dislike the use of black-box routines, we need to ask you to accept this one, since it would take us too far afield to cover its necessary background material here. The algorithm is very stable, and it is very unusual for it ever to misbehave. Most of the concepts that enter the algorithm (Householder reduction to bidiagonal form, diagonalization by *QR* procedure with shifts) will be discussed further in Chapter 11.

As we did for *LU* decomposition, we encapsulate the singular value decomposition and also the methods that depend on it into an object, *SVD*. We give its declaration here. The rest of this section will give the details on how to use it.

```
struct SVD { svd.h
    Object for singular value decomposition of a matrix A, and related functions.
    Int m,n;
    MatDoub u,v; The matrices U and V.
    VecDoub w; The diagonal matrix W.
    Doub eps, tsh;
    SVD(MatDoub_I &a) : m(a.nrows()), n(a.ncols()), u(a), v(n,n), w(n) {
        Constructor. The single argument is A. The SVD computation is done by decompose, and
        the results are sorted by reorder.
        eps = numeric_limits<Doub>::epsilon();
        decompose();
        reorder();
        tsh = 0.5*sqrt(m+n+1.)*w[0]*eps; Default threshold for nonzero singular
        values.
    }

    void solve(VecDoub_I &b, VecDoub_O &x, Doub thresh);
    void solve(MatDoub_I &b, MatDoub_O &x, Doub thresh);
    Solve with (apply the pseudoinverse to) one or more right-hand sides.

    Int rank(Doub thresh); Quantities associated with the range and
    Int nullity(Doub thresh); nullspace of A.
    MatDoub range(Doub thresh);
    MatDoub nullspace(Doub thresh);

    Doub inv_condition() { Return reciprocal of the condition num-
        return (w[0] <= 0. || w[n-1] <= 0.) ? 0. : w[n-1]/w[0]; ber of A.
    }

    void decompose(); Functions used by the constructor.
    void reorder();
    Doub pythag(const Doub a, const Doub b);
};
```

2.6.1 Range, Nullspace, and All That

Consider the familiar set of simultaneous equations

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b} \quad (2.6.5)$$

where **A** is an $M \times N$ matrix, and **x** and **b** are vectors of dimension N and M respectively. Equation (2.6.5) defines **A** as a linear mapping from an N -dimensional vector space to (generally) an M -dimensional one. But the map *might* be able to reach only a lesser-dimensional subspace of the full M -dimensional one. That subspace is called the *range* of **A**. The dimension of the range is called the *rank* of **A**. The rank

of $\mathbf{A}$ is equal to its number of linearly independent columns, and also (perhaps less obviously) to its number of linearly independent rows. If $\mathbf{A}$ is not identically zero, its rank is at least 1, and at most $\min(M, N)$.

Sometimes there are nonzero vectors $\mathbf{x}$ that are mapped to zero by $\mathbf{A}$, that is, $\mathbf{A} \cdot \mathbf{x} = 0$. The space of such vectors (a subspace of the N -dimensional space that $\mathbf{x}$ lives in) is called the *nullspace* of $\mathbf{A}$, and its dimension is called $\mathbf{A}$'s *nullity*. The nullity can have any value from zero to N . The *rank-nullity theorem* states that, for any $\mathbf{A}$, the rank plus the nullity is N , the number of columns.

An important special case is $M = N$, so the $\mathbf{A}$ is square, $N \times N$. If the rank of $\mathbf{A}$ is N , its maximum possible value, then $\mathbf{A}$ is nonsingular and invertible: $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ has a unique solution for any $\mathbf{b}$, and only the zero vector is mapped to zero. This is a case where *LU* decomposition (§2.3) is the preferred solution method for $\mathbf{x}$. However, if $\mathbf{A}$ has rank less than N (i.e., has nullity greater than zero), then two things happen: (i) most right-hand side vectors $\mathbf{b}$ yield no solution, but (ii) some have multiple solutions (in fact a whole subspace of them). We consider this situation further, below.

What has all this to do with singular value decomposition? SVD explicitly constructs orthonormal bases for the nullspace and range of a matrix! Specifically, the columns of $\mathbf{U}$ whose same-numbered elements w_j are *nonzero* are an orthonormal set of basis vectors that span the range; the columns of $\mathbf{V}$ whose same-numbered elements w_j are *zero* are an orthonormal basis for the nullspace. Our SVD object has methods that return the rank or nullity (integers), and also the range and nullspace, each of these packaged as a matrix whose columns form an orthonormal basis for the respective subspace.

```
svd.h  Int SVD::rank(Doub thresh = -1.) {
Return the rank of  $\mathbf{A}$ , after zeroing any singular values smaller than thresh. If thresh is
negative, a default value based on estimated roundoff is used.
    Int j,nr=0;
    tsh = (thresh >= 0. ? thresh : 0.5*sqrt(m+n+1.)*w[0]*eps);
    for (j=0;j<n;j++) if (w[j] > tsh) nr++;
    return nr;
}

Int SVD::nullity(Doub thresh = -1.) {
Return the nullity of  $\mathbf{A}$ , after zeroing any singular values smaller than thresh. Default value as
above.
    Int j,nn=0;
    tsh = (thresh >= 0. ? thresh : 0.5*sqrt(m+n+1.)*w[0]*eps);
    for (j=0;j<n;j++) if (w[j] <= tsh) nn++;
    return nn;
}

MatDoub SVD::range(Doub thresh = -1.){
Give an orthonormal basis for the range of  $\mathbf{A}$  as the columns of a returned matrix. thresh as
above.
    Int i,j,nr=0;
    MatDoub rnge(m,rank(thresh));
    for (j=0;j<n;j++) {
        if (w[j] > tsh) {
            for (i=0;i<m;i++) rnge[i][nr] = u[i][j];
            nr++;
        }
    }
    return rnge;
}
```